

Cloud Computing - Setup e Dev

Ronierison Maciel

Fevereiro 2025



Quem sou eu?



Nome: Roni

Formação: Mestre em Ciência da Computação

Ocupação: Pesquisador, Professor e Escovador de bits

Hobbies: Jogar cartas, ficar com a família

Interesses: Carros, DevSec, DS e ML

Email: ronierison.maciел@pe.senac.br

GitHub: <https://github.com/0x03c1>

Conteúdo

- 1 Conceitos Iniciais
- 2 Conceitos de Infraestrutura na Nuvem
- 3 Provedores de Nuvem
- 4 Infraestrutura como Código (IaC)
- 5 Conceito de Containers
- 6 Monitoramento e logging

Objetivos:

- Compreender o conceito de computação em nuvem e sua importância.
- Diferenciar os modelos de serviço em nuvem (IaaS, PaaS, SaaS, Serverless).
- Explorar exemplos práticos de uso da computação em nuvem.

Quem já usou algum serviço na nuvem sem perceber?

Exemplos comuns:

- Google Drive
- Netflix
- Spotify
- Gmail

O que é computação em nuvem?

Definição: Computação em nuvem é um modelo de fornecimento de serviços computacionais sob demanda através da internet.

Benefícios:

- Escalabilidade
- Elasticidade
- Redução de custos

Benefícios da Computação em Nuvem



IaaS (Infrastructure as a Service)

- Fornece infraestrutura virtualizada.

PaaS (Platform as a Service)

- Oferece plataforma para desenvolvimento.

SaaS (Software as a Service)

- Aplicações completas executadas na nuvem.

Serverless computing

- Execução sob demanda sem gerenciamento de servidores.

Pergunta: Como a computação em nuvem pode impactar sua carreira no futuro?

Discussão:

- Crescimento da demanda por especialistas em nuvem.
- Possibilidades de trabalho remoto.
- Oportunidades para startups e inovação.

Pesquise um caso real de migração para a nuvem:

- Escolha uma empresa ou projeto que adotou a nuvem.
- Identifique benefícios e desafios enfrentados.
- Prepare um resumo para discussão na próxima aula.

Resumo da aula:

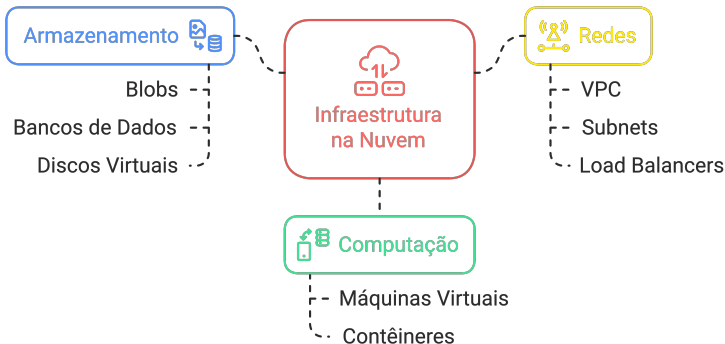
- Conceitos e importância da computação em nuvem.
- Modelos de serviço (IaaS, PaaS, SaaS, Serverless).
- Exemplos práticos e atividades interativas.

Objetivos:

- Compreender os conceitos fundamentais de infraestrutura na nuvem.
- Explorar os componentes básicos de infraestrutura em cloud computing.
- Apresentar casos reais de implementação.

Infraestrutura na nuvem?

Infraestrutura na Nuvem: Componentes e Relações



O que é infraestrutura na nuvem?

Definição: Conjunto de servidores, redes, armazenamento e serviços virtuais fornecidos sob demanda por um provedor de nuvem.

Principais componentes:

- Computação: Instâncias de máquinas virtuais.
- Armazenamento: Blobs, bancos de dados e discos virtuais.
- Redes: VPC, Subnets, Load Balancers.

Benefícios da infraestrutura na nuvem

- Elasticidade: Ajuste automático de recursos conforme a demanda.
- Redução de custos: Pagamento sob demanda.
- Disponibilidade global: Acesso a data centers distribuídos.
- Automação: Gerenciamento via APIs e scripts.

Tipos de infraestrutura na nuvem

Infraestrutura Compartilhada

- Vários clientes compartilham os mesmos recursos físicos.
- Exemplo: AWS EC2 Shared Instances.

Infraestrutura Dedicada

- Recursos exclusivos para um único cliente.
- Exemplo: AWS Dedicated Hosts.

Infraestrutura Híbrida

- Combina infraestrutura local e nuvem pública.
- Exemplo: Azure Hybrid Cloud.

Atividade: Escolha o melhor modelo

Instrução:

- Dividir a turma em grupos.
- Cada grupo recebe um caso de uso real.
- Decidir se a melhor solução é pública, privada ou híbrida.
- Apresentar as justificativas.

Computação:

- Máquinas virtuais (EC2, GCE, Azure VMs).
- Containers (Docker, Kubernetes).

Armazenamento:

- Blocos: EBS, Persistent Disks.
- Objetos: S3, Google Cloud Storage.
- Bancos de dados: RDS, Firestore.

Redes:

- VPC, VPNs, Load Balancers, Firewalls.

Demonstração: Criando uma instância EC2

Passos:

- 1 Acessar o console da AWS.
- 2 Criar uma instância EC2.
- 3 Escolher sistema operacional (Linux ou Windows).
- 4 Configurar tipo da máquina e rede.
- 5 Acessar via SSH.

Instrução:

- Cada aluno cria uma instância EC2 no AWS Free Tier.
- Configura e acessa a instância via SSH.
- Explora comandos básicos de administração.

Pergunta: Como a infraestrutura na nuvem impacta o desenvolvimento de software e a escalabilidade?

Discussão:

- Redução da necessidade de hardware local.
- Possibilidade de escalar aplicações globalmente.
- Facilidade de automação e integração com DevOps.

Pesquisa:

- Escolha um serviço de infraestrutura na nuvem (ex: AWS EC2, Google Compute Engine, Azure VMs).
- Descreva seu funcionamento e principais casos de uso.
- Apresente um resumo na próxima aula.

Resumo da aula:

- Conceitos fundamentais de infraestrutura na nuvem.
- Modelos de infraestrutura (compartilhada, dedicada, híbrida).
- Prática com instâncias EC2.

Objetivos:

- Compreender os diferentes modelos de serviço em nuvem.
- Identificar as diferenças entre IaaS, PaaS, SaaS e Serverless.
- Explorar casos de uso de cada modelo.

O que são modelos de serviço na nuvem?

Definição: Os modelos de serviço na nuvem definem como os recursos são disponibilizados e gerenciados pelos provedores.

Principais modelos:

- IaaS (Infrastructure as a Service)
- PaaS (Platform as a Service)
- SaaS (Software as a Service)
- Serverless Computing

Características:

- Fornece infraestrutura virtualizada sob demanda.
- Usuário gerencia sistema operacional, rede e armazenamento.
- Alta flexibilidade e escalabilidade.

Exemplos:

- AWS EC2
- Google Compute Engine
- Microsoft Azure Virtual Machines

Modelo de serviço na nuvem - IaaS



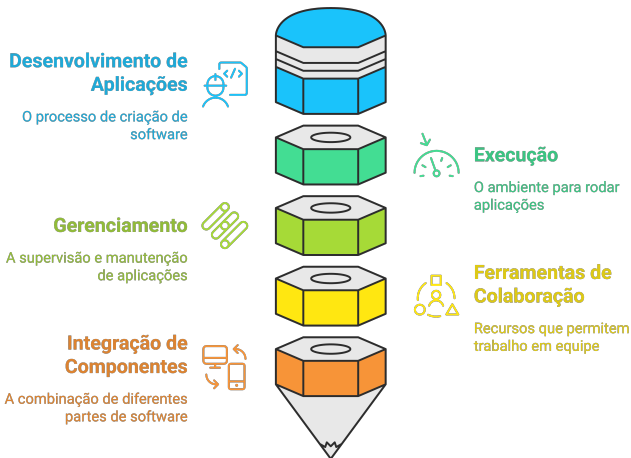
Características:

- Ambiente completo para desenvolvimento e hospedagem.
- Infraestrutura gerenciada pelo provedor.
- Ideal para desenvolvimento ágil de aplicações.

Exemplos:

- AWS Elastic Beanstalk
- Google App Engine
- Heroku

Compreendendo o PaaS



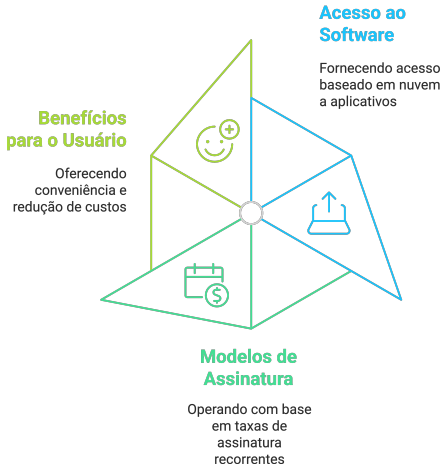
Características:

- Aplicações completas hospedadas e gerenciadas na nuvem.
- O usuário final acessa via navegador ou API.
- Modelo mais comum para usuários finais.

Exemplos:

- Google Drive
- Microsoft Office 365
- Dropbox

Compreendendo o SaaS



Características:

- Execução de código sem necessidade de gerenciar servidores.
- Cobrança apenas pelo tempo de execução.
- Alta escalabilidade e eficiência de custo.

Exemplos:

- AWS Lambda
- Google Cloud Functions
- Azure Functions

Modelo de serviço na nuvem - Serverless

O que é Serverless



Comparação entre os modelos

Modelo	Controle	Escalabilidade	Complexidade	Exemplo
IaaS	Alto	Alta	Alta	AWS EC2
PaaS	Médio	Alta	Média	Google App Engine
SaaS	Baixo	Alta	Baixa	Dropbox
Serverless	Baixo	Automática	Baixa	AWS Lambda

Instrução:

- Dividir a turma em grupos.
- Fornecer uma lista de serviços em nuvem.
- Cada grupo deve classificar os serviços em IaaS, PaaS, SaaS ou Serverless.
- Apresentar resultados e promover discussão.

Arquitetura em Nuvem:

- Utiliza AWS para armazenar e processar conteúdo.
- Usa IaaS (EC2) para servidores escaláveis.
- Usa PaaS para serviços internos.
- Usa Serverless para automação e eventos.

Demonstração: Criando um serviço PaaS

Passos:

- 1 Acessar AWS Elastic Beanstalk.
- 2 Criar uma aplicação web simples.
- 3 Configurar ambiente (Python, Node.js, Java).
- 4 Implantar e testar a aplicação.

Pergunta: Qual modelo de serviço é mais adequado para diferentes tipos de empresas?

Discussão:

- Startups costumam usar PaaS ou Serverless pela agilidade.
- Grandes empresas podem preferir IaaS para maior controle.
- Usuários finais utilizam SaaS diariamente.

Pesquisa:

- Escolha um serviço SaaS, PaaS ou IaaS.
- Explique seu funcionamento e benefícios.
- Apresente um resumo de no máximo de 2 páginas 5 min.

Resumo da aula:

- Diferenças entre IaaS, PaaS, SaaS e Serverless.
- Casos de uso reais para cada modelo.
- Atividade prática e demonstração.

Objetivos:

- Apresentar os principais provedores de nuvem.
- Compreender as diferenças entre AWS, Azure e GCP.
- Explorar serviços oferecidos por cada provedor.

O que são provedores de nuvem?

O que são provedores de nuvem?

Empresas que fornecem infraestrutura, plataforma e software como serviço na nuvem.

Quais são os principais provedores?

AWS,
Microsoft Azure e
Google Cloud Platform.



Amazon Web Services (AWS)



Características:

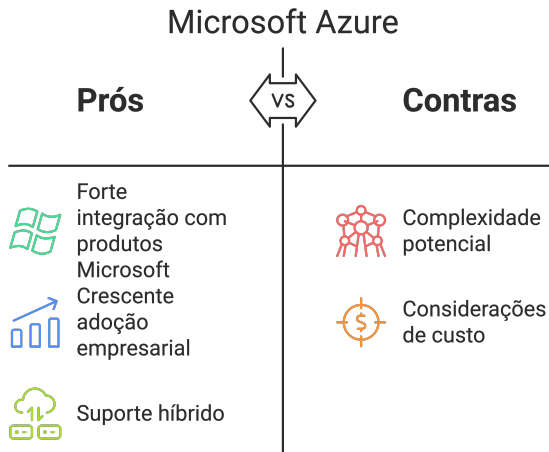
- Maior provedor de nuvem do mundo.
- Ampla variedade de serviços (computação, banco de dados, IA).
- Grande comunidade e suporte.

Principais serviços:

- Computação: EC2, Lambda
- Armazenamento: S3, EBS
- Banco de Dados: RDS, DynamoDB
- Redes: VPC, CloudFront

Serviços de Nuvem da AWS





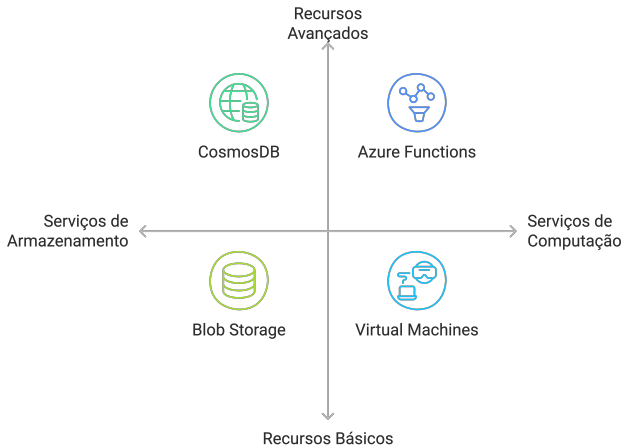
Características:

- Forte integração com produtos Microsoft.
- Crescente adoção empresarial.
- Suporte híbrido (on-premise + cloud).

Principais serviços:

- Computação: Virtual Machines, Azure Functions
- Armazenamento: Blob Storage, Files
- Banco de Dados: SQL Database, CosmosDB
- Redes: Virtual Network, Traffic Manager

Serviços e Recursos do Microsoft Azure



Google Cloud Platform (GCP)



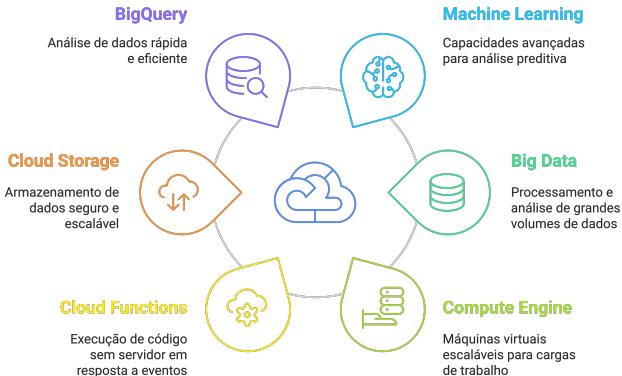
Características:

- Forte em Machine Learning e Big Data.
- Rede global de baixa latência.
- Modelos de preços competitivos.

Principais serviços:

- Computação: Compute Engine, Cloud Functions
- Armazenamento: Cloud Storage, Filestore
- Banco de Dados: BigQuery, Firestore
- Redes: VPC, Cloud CDN

Componentes da Plataforma Google Cloud



Comparação entre provedores

Provedor	Vantagem Principal	Foco/Melhor Uso	Exemplo de Cliente
AWS	Maior variedade de serviços	Empresas de todos os tamanhos	Netflix
Azure	Integração com Microsoft	Corporações e ambientes híbridos	BMW
GCP	Machine Learning e Big Data	Startups e análise de dados	Spotify

Instrução:

- Dividir os alunos em grupos.
- Cada grupo deve escolher um provedor (AWS, Azure ou GCP).
- Criar um caso de uso real para o provedor escolhido.
- Apresentar resultados para a turma.

Infraestrutura da Netflix na nuvem:

- Hospedagem na AWS.
- Uso de Auto Scaling para garantir disponibilidade.
- CDN CloudFront para reduzir latência de streaming.
- Banco de dados DynamoDB para armazenar preferências dos usuários.

Demonstração: Criando uma Conta na AWS

Passos:

- 1 Acessar <https://aws.amazon.com> e clicar em "Criar Conta".
- 2 Fornecer informações pessoais e método de pagamento.
- 3 Escolher o plano Free Tier.
- 4 Fazer login e explorar o console AWS.

Instrução:

- Criar uma conta gratuita na AWS, Azure ou GCP.
- Explorar os serviços básicos oferecidos.
- Compartilhar a experiência na próxima aula.

Pergunta: Qual provedor seria mais adequado para diferentes cenários de negócio?

Discussão:

- Startups buscam GCP para análise de dados.
- Corporações utilizam Azure para integração com Windows.
- Empresas de streaming optam por AWS pela escalabilidade.

Pesquisa:

- Escolha uma empresa que utilize um provedor de nuvem.
- Explique os serviços que a empresa usa e os benefícios.
- Apresente um resumo na próxima aula.

Resumo:

- AWS, Azure e GCP são os principais provedores de nuvem.
- Cada um tem vantagens distintas de acordo com o caso de uso.
- Experimentação prática com criação de contas.

Objetivos:

- Entender o conceito de Infraestrutura como Código (IaC).
- Explorar ferramentas populares: Terraform e AWS CloudFormation.
- Demonstrar um exemplo prático de automação de infraestrutura.

O que é infraestrutura como código (IaC)?

Definição: Prática que permite gerenciar e provisionar infraestrutura através de código, em vez de configurações manuais.

Benefícios:

- Automação e padronização.
- Redução de erros humanos.
- Facilidade para replicar ambientes.
- Controle de versões da infraestrutura.

Ferramentas de infraestrutura como código

Principais ferramentas:

- Terraform (HashiCorp)
- AWS CloudFormation
- Ansible (Red Hat)
- Pulumi

Destaques:

- **Terraform:** Multicloud e código declarativo.
- **CloudFormation:** Específico para AWS.
- **Ansible:** Configuração e automação de servidores.

Características:

- Código declarativo em HCL (HashiCorp Configuration Language).
- Compatível com AWS, Azure, GCP e outros.
- Permite planejamento e execução da infraestrutura.

Principais Comandos:

- `terraform init` - Inicializa o ambiente.
- `terraform plan` - Exibe mudanças planejadas.
- `terraform apply` - Aplica configurações.
- `terraform destroy` - Remove infraestrutura.

Características:

- Usa arquivos YAML ou JSON para definir recursos.
- Gerencia o provisionamento de serviços AWS.
- Suporte a pilhas (*stacks*) para gerenciar grupos de recursos.

Principais conceitos:

- **Templates:** Definem a infraestrutura.
- **Stacks:** Implementam e gerenciam recursos.
- **Rollback:** Reverte mudanças em caso de falha.

Comparação: Terraform vs CloudFormation

Característica	Terraform	CloudFormation
Suporte Multicloud	Sim	Não
Linguagem	HCL	YAML/JSON
Gerenciamento de Estado	Sim	Sim
Automação Nativa AWS	Parcial	Completa

Atividade: Escolhendo a melhor ferramenta

Instrução:

- Dividir a turma em grupos.
- Cada grupo deve escolher entre Terraform ou CloudFormation.
- Criar um cenário de uso e justificar a escolha.
- Apresentar os resultados.

Demonstração: Criando uma Infraestrutura com Terraform

Passos:

- 1 Instalar Terraform (<https://www.terraform.io>).
- 2 Criar um arquivo `main.tf`.
- 3 Definir um recurso EC2 na AWS.
- 4 Executar os comandos `init`, `plan` e `apply`.

Exemplo: Criando uma instância EC2 com Terraform

```
1 provider "aws" {  
2     region = "us-east-1"  
3 }  
4  
5 resource "aws_instance" "web" {  
6     ami          = "ami-0c55b159cbfafa1f0"  
7     instance_type = "t2.micro"  
8 }
```

Instrução:

- Criar um arquivo Terraform para provisionar um recurso na AWS.
- Executar os comandos Terraform para criar a infraestrutura.
- Compartilhar a experiência na próxima aula.

Pergunta: Como a Infraestrutura como Código melhora o gerenciamento de ambientes na nuvem?

Discussão:

- Automação reduz esforço manual.
- Controle de versões facilita auditoria e rollback.
- Infraestrutura replicável agiliza testes e produção.

Pesquisa:

- Escolha uma empresa que use IaC.
- Explique quais ferramentas são utilizadas e os benefícios obtidos.
- Apresente um resumo na próxima aula.

Resumo da aula:

- Conceitos de Infraestrutura como Código e suas vantagens.
- Uso de Terraform para ambientes multicloud.
- Uso de CloudFormation para automação na AWS.
- Exemplo prático de criação de uma instância EC2.

Objetivos:

- Compreender o conceito de containers e sua importância.
- Explorar Docker para criação e gerenciamento de containers.
- Entender Kubernetes para orquestração de containers em larga escala.

O que são Containers?

Definição: Containers são unidades leves e portáteis que empacotam uma aplicação e suas dependências.

Benefícios:

- Consistência entre ambientes (desenvolvimento, teste, produção).
- Melhor utilização de recursos em comparação com VMs.
- Facilidade de escalabilidade e replicação.

Características:

- Facilita a criação, empacotamento e execução de aplicações em containers.
- Imagens Docker permitem portabilidade total.
- Gerenciamento simplificado via Docker CLI e Docker Compose.

Principais comandos:

- `docker pull` — Baixa uma imagem do Docker Hub.
- `docker run` — Executa um container.
- `docker ps` — Lista containers em execução.
- `docker stop` — Para um container.

Demonstração: Criando um container com Docker

Passos:

- 1 Instalar Docker (<https://www.docker.com>).
- 2 Baixar uma imagem oficial do Nginx: `docker pull nginx`
- 3 Rodar um container Nginx: `docker run -d -p 8080:80 nginx`
- 4 Acessar no navegador: `http://localhost:8080`

Características:

- Gerencia clusters de containers em escala.
- Permite balanceamento de carga e alta disponibilidade.
- Suporte para autoescalonamento e autorrecuperação.

Principais componentes:

- **Pods:** Unidades básicas de execução.
- **Deployments:** Controlam atualizações e réplicas de pods.
- **Services:** Expõem os pods para comunicação.

Comparação Docker vs Kubernetes

Característica	Docker	Kubernetes
Gerenciamento de Containers	Sim	Sim
Orquestração	Não	Sim
Escalabilidade	Baixa	Alta
Uso em produção	Limitado	Padrão

Atividade: Escolhendo a melhor solução

Instrução:

- Divida os alunos em grupos.
- Cada grupo escolhe entre Docker ou Kubernetes para um caso real.
- Apresentar resultados com justificativas.

Demonstração: Criando um Cluster Kubernetes

Passos:

- 1 Instalar minikube e kubectl.
- 2 Iniciar um cluster local: `minikube start`
- 3 Criar um pod com Nginx: `kubectl run nginx -image=nginx -port=80`
- 4 Expor o pod: `kubectl expose pod nginx -type=NodePort -port=80`
- 5 Acessar: `minikube service nginx`

Instrução:

- Criar um container Docker com uma aplicação simples.
- Implantar a aplicação em um cluster Kubernetes local (Minikube).
- Compartilhar a experiência na próxima aula.

Pergunta: Quando usar Docker e quando usar Kubernetes?

Discussão:

- Docker é ideal para desenvolvimento local e aplicações menores.
- Kubernetes é fundamental para aplicações distribuídas, escaláveis e de alta disponibilidade.

Pesquisa:

- Escolha uma empresa que use Kubernetes em produção.
- Explique os benefícios e desafios enfrentados.
- Apresente um resumo na próxima aula.

Resumo da aula:

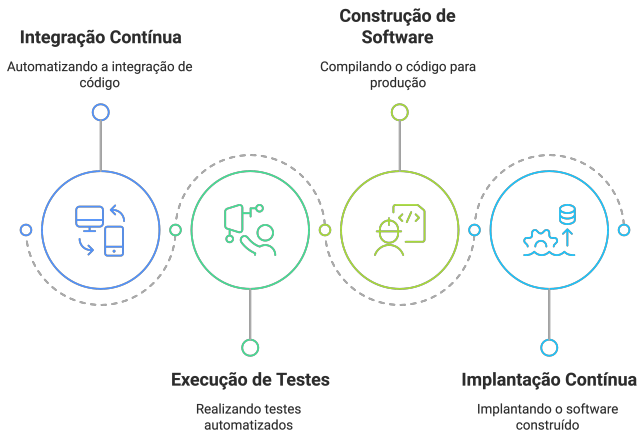
- Containers são fundamentais para portabilidade e escalabilidade.
- Docker facilita a criação e gerenciamento de containers.
- Kubernetes orquestra e gerencia clusters de containers.

Objetivos da Aula:

- Compreender o conceito de CI/CD e sua importância.
- Explorar GitHub Actions e Jenkins para automação de deploys.
- Criar um pipeline CI/CD simples na prática.

O que é CI/CD?

Processo CI/CD



O que é CI/CD?

Definição: CI/CD (Continuous Integration / Continuous Deployment) é um conjunto de práticas que automatizam testes, builds e deploys de software.

Benefícios:

- Redução de erros humanos no processo de deploy.
- Entrega mais rápida e confiável de novas versões.
- Maior qualidade do software devido à automação de testes.

Características:

- Plataforma integrada ao GitHub para automação de workflows.
- Utiliza YAML para definir jobs e ações.
- Oferece runners gratuitos para repositórios públicos.

Principais Conceitos:

- **Workflows:** Definem o pipeline de CI/CD.
- **Jobs:** Conjuntos de tarefas que podem rodar em paralelo ou em sequência.
- **Actions:** Conjuntos de comandos ou scripts reutilizáveis.

Exemplo de Workflow no GitHub Actions

```
1 name: CI Pipeline
2 on: push
3 jobs:
4   build:
5     runs-on: ubuntu-latest
6     steps:
7       - name: Checkout do código
8         uses: actions/checkout@v2
9
10      - name: Configurar ambiente Python
11        uses: actions/setup-python@v2
12        with:
13          python-version: '3.8'
14
15      - name: Instalar dependências
16        run: pip install -r requirements.txt
17
18      - name: Rodar testes
19        run: pytest
```

Características:

- Ferramenta open-source para automação de CI/CD.
- Suporte a múltiplos ambientes e amplo ecossistema de plugins.
- Permite criação de pipelines declarativos em formato de script (Groovy).

Principais Conceitos:

- **Jobs:** Unidades básicas de execução dentro do Jenkins.
- **Pipelines:** Sequência de tarefas automatizadas (build, teste, deploy).
- **Nodes:** Máquinas ou agentes onde os jobs são executados.

Comparação: GitHub Actions vs Jenkins

Característica	GitHub Actions	Jenkins
Integração com GitHub	Alta (nativo)	Parcial
Infraestrutura	Nuvem (SaaS)	On-premise ou Cloud
Configuração	YAML	Script Groovy (Jenkinsfile)
Extensibilidade (Plugins)	Limitada	Grande ecossistema

Reflexão: Quando usar cada um?

Discussão:

- **GitHub Actions:** Melhor para repositórios hospedados no GitHub e projetos que buscam praticidade.
- **Jenkins:** Ideal para integrações complexas, maior controle do ambiente e uso on-premise.
- Contextualizar com cenários reais: tamanho da equipe, infraestrutura disponível e necessidades de escalabilidade.

Atividade: Escolhendo a Melhor Solução

Instrução:

- Dividir a turma em grupos.
- Cada grupo escolhe entre GitHub Actions ou Jenkins para um projeto real.
- Justificar a escolha com base em cenários (tamanho da equipe, tipo de hospedagem, ferramentas já utilizadas).
- Apresentar os resultados ao final da aula.

Demonstração: Criando um Pipeline CI/CD

Passos:

- 1 Criar um repositório no GitHub.
- 2 Adicionar um arquivo `.github/workflows/ci.yml` (para GitHub Actions).
- 3 Configurar o pipeline para rodar testes e/ou fazer build.
- 4 Fazer `git push` para acionar a pipeline.
- 5 Verificar o status do workflow no GitHub Actions (Aba *Actions*).

Instrução:

- **Opção 1 (GitHub Actions):** Criar um pipeline para rodar testes unitários e gerar build de um projeto simples em Python ou Node.js.
- **Opção 2 (Jenkins):** Instalar o Jenkins localmente (ou usar Docker), criar um Jenkinsfile e configurar etapas de build e teste.
- Compartilhar o link do repositório ou screenshots dos resultados na próxima aula.

Resumo da Aula:

- CI/CD automatiza e agiliza o desenvolvimento de software.
- GitHub Actions fornece integração nativa com o GitHub.
- Jenkins é uma solução open-source altamente extensível.
- Escolha da ferramenta depende do ambiente, escala e requisitos do projeto.

A definir

Objetivos da Aula:

- Compreender a importância do monitoramento e logging em aplicações na nuvem.
- Explorar ferramentas populares como CloudWatch, Prometheus e ELK Stack.
- Implementar um exemplo prático de monitoramento.

Por que Monitoramento e Logging?

Motivações:

- Diagnóstico rápido de problemas e falhas.
- Otimização de performance e uso de recursos.
- Compliance e segurança de aplicações na nuvem.

Tipos de Monitoramento:

- Infraestrutura (CPU, memória, rede).
- Aplicação (tempo de resposta, erros).
- Logging (registro de eventos e auditoria).

Características:

- Serviço da AWS para monitoramento de métricas e logs.
- Coleta e analisa dados de serviços AWS e aplicações personalizadas.
- Integração com alarmes para notificações automáticas.

Principais Recursos:

- Logs Insights — Análise de logs em tempo real.
- Métricas Customizadas — Definição de métricas próprias.
- Alarmes — Gatilhos para eventos críticos.

Demonstração: Criando um Alarme no CloudWatch

Passos:

- 1 Acessar o console AWS CloudWatch.
- 2 Criar uma nova métrica para monitorar CPU de uma instância EC2.
- 3 Definir um alarme para uso acima de 80%.
- 4 Configurar notificação via SNS.
- 5 Simular carga e verificar acionamento do alarme.

Características:

- Coleta métricas de sistemas distribuídos.
- Armazena dados em séries temporais.
- Oferece alertas e consultas avançadas (PromQL).

Principais Componentes:

- Exporters — Capturam métricas de aplicações.
- Alertmanager — Gatilhos para notificações.
- Grafana — Visualização e dashboards interativos.

Demonstração: Instalando Prometheus

Passos:

- 1 Baixar e instalar Prometheus.
- 2 Configurar um `prometheus.yml` para coletar métricas.
- 3 Rodar Prometheus localmente.
- 4 Acessar `http://localhost:9090` e visualizar métricas.

O que é ELK?

- Elasticsearch — Armazena e indexa logs.
- Logstash — Coleta e processa logs de múltiplas fontes.
- Kibana — Interface para visualização e análise de logs.

Benefícios:

- Análise detalhada de eventos e erros.
- Identificação rápida de padrões e anomalias.
- Integração com diversas fontes de logs.

Demonstração: Configurando ELK Stack

Passos:

- 1 Baixar e instalar Elasticsearch, Logstash e Kibana.
- 2 Configurar Logstash para coletar logs de uma aplicação.
- 3 Enviar logs para Elasticsearch.
- 4 Visualizar os logs em Kibana.

Comparação: CloudWatch vs Prometheus vs ELK

Característica	CloudWatch	Prometheus	ELK
Escopo	AWS	Multi-plataforma	Logs centralizados
Armazenamento	Cloud	Local ou Cloud	Elasticsearch
Alertas	Sim	Sim (Alertmanager)	Sim (via Kibana ou Beats)
Visualização	Console AWS	Grafana	Kibana

Atividade: Escolhendo a Melhor Solução

Instrução:

- Divida os alunos em grupos.
- Cada grupo escolhe uma ferramenta para um caso real (AWS, on-premise, microserviços etc.).
- Justificar a escolha, apontando vantagens e desvantagens.
- Apresentar os resultados no final da aula.

Instrução:

- Criar um alerta no CloudWatch para monitorar um recurso AWS.
- Configurar Prometheus para coletar métricas de uma aplicação local.
- Enviar logs de uma aplicação para Elasticsearch e visualizar no Kibana.
- Documentar e compartilhar a experiência na próxima aula.

Pergunta: Qual a importância de um bom monitoramento em ambientes escaláveis?

Discussão:

- Permite reação rápida a falhas e incidentes.
- Otimiza o uso de recursos e melhora a performance.
- Aumenta a confiabilidade e segurança do ambiente.

Pesquisa:

- Escolha uma empresa que use monitoramento na nuvem.
- Explique quais ferramentas são utilizadas e os benefícios.
- Apresente um resumo na próxima aula.

Resumo da aula:

- Monitoramento e logging são essenciais para aplicações na nuvem.
- CloudWatch é ideal para ambientes AWS.
- Prometheus e ELK oferecem soluções robustas para múltiplos cenários.

Objetivos da Aula:

- Consolidar os conhecimentos aplicando na prática.
- Criar um ambiente funcional na nuvem.
- Instanciar e configurar recursos essenciais (EC2, S3, Security Group).

Componentes:

- Instância EC2 rodando Linux Ubuntu.
- Bucket S3 para armazenamento.
- Security Group permitindo SSH e HTTP.
- Configuração de chave de acesso (key pair).

Antes de começar, o aluno deve:

- Ter uma conta na AWS com Free Tier ativa.
- Ter acesso ao console da AWS.
- Ter uma chave SSH para acesso à instância.

Passo 1: Criar a Instância EC2

Instruções:

- 1 Acessar o Console AWS.
- 2 Ir até EC2 > Launch Instance.
- 3 Nome: ambiente-pratico
- 4 AMI: Ubuntu Server 22.04 LTS
- 5 Tipo: t2.micro
- 6 Par de chaves: Criar ou usar existente.
- 7 Configurar Security Group:
 - Liberar porta 22 (SSH) e 80 (HTTP).
- 8 Lançar a instância.

Passo 2: Conectar via SSH

Instruções:

- Obter o IP público da instância.
- Usar o terminal com o comando:
- `ssh -i "chave.pem" ubuntu@IP_Público`
- Acessar e atualizar os pacotes:
- `sudo apt update && sudo apt upgrade -y`

Passo 3: Instalar um Servidor Web

Objetivo:

- Instalar e testar um servidor web básico (Apache).

Comandos:

- `sudo apt install apache2 -y`
- Verificar status: `sudo systemctl status apache2`
- Acessar via navegador: `http://IP_Público`

Passo 4: Criar e Configurar um Bucket S3

Instruções:

- 1 Acessar S3 no Console AWS.
- 2 Criar novo bucket com nome único.
- 3 Desativar bloqueio de acesso público (apenas para testes).
- 4 Fazer upload de um arquivo.
- 5 Acessar URL pública do arquivo.

Tarefa:

- Em dupla, cada aluno configura seu ambiente completo na AWS.
- Instância EC2 acessível via navegador.
- Upload de arquivo no S3 e compartilhamento de link.
- Validar com outro grupo.

Pergunta:

- Qual foi o maior desafio ao configurar o ambiente?
- Como essa prática ajuda no entendimento de nuvem?

Discussão:

- A importância da prática para consolidar teoria.
- Como automatizar isso com Terraform nas próximas aulas.

Instrução:

- Documentar os passos da configuração em Markdown.
- Incluir prints das telas, comandos e links dos recursos.
- Subir o arquivo no GitHub pessoal com título “Ambiente na Nuvem”.

Resumo da aula:

- Criamos um ambiente real na AWS.
- Usamos EC2, S3 e regras de segurança.
- Prática essencial para fixar os conceitos de nuvem.

Próxima aula: Segurança na Nuvem: Controle de acesso, criptografia e conformidade.

Objetivos da Aula:

- Entender os principais riscos e boas práticas de segurança na nuvem.
- Trabalhar com controle de acesso, criptografia e conformidade.
- Aplicar configurações básicas de segurança na AWS.

Exemplos comuns:

- Credenciais expostas em repositórios públicos.
- Buckets S3 mal configurados (acesso público indevido).
- Ataques DDoS e escalonamento de privilégios.
- Falta de criptografia em dados sensíveis.

IAM (Identity and Access Management):

- Permite gerenciar usuários, grupos e permissões na nuvem.
- Princípio do menor privilégio: cada entidade só tem acesso ao necessário.

Boas práticas:

- Não usar root para tarefas do dia a dia.
- Criar usuários individuais e aplicar políticas específicas.
- Usar autenticação multifator (MFA).

Demonstração: Criando Usuário com Acesso Limitado

Passos:

- 1 Acessar AWS IAM.
- 2 Criar um novo usuário (ex: aluno-cloud).
- 3 Atribuir permissão apenas de leitura em EC2 e S3.
- 4 Ativar MFA (opcional).
- 5 Testar o login com permissões limitadas.

Criptografia em repouso:

- Protege dados armazenados (S3, RDS, EBS).
- AWS KMS (Key Management Service) gerencia as chaves.

Criptografia em trânsito:

- Protege dados em movimento usando TLS/HTTPS.
- Deve ser habilitada para APIs, aplicações e acesso a serviços.

Demonstração: Habilitando Criptografia no S3

Passos:

- 1 Acessar S3 > Bucket criado.
- 2 Ir na aba “Properties” > Encryption.
- 3 Ativar criptografia com chave padrão do S3 ou usar o AWS KMS.
- 4 Fazer upload de novo arquivo e validar.

O que é conformidade?

- Garantia de que as práticas seguem leis, normas e políticas.

Exemplos de normas:

- LGPD (Brasil), GDPR (Europa), HIPAA (EUA), ISO 27001.

Ferramentas AWS:

- AWS Artifact: Documentação de conformidade.
- AWS Config: Avalia conformidade com regras.
- AWS Security Hub: Centraliza alertas de segurança.

Tarefa prática:

- Criar um usuário IAM com permissões limitadas.
- Ativar criptografia em bucket S3.
- Documentar as etapas em um arquivo Markdown.
- Compartilhar o link do GitHub com a documentação.

Pergunta:

- Como você garantiria a segurança da sua aplicação na nuvem?

Discussão:

- Controle de acesso é o primeiro passo.
- Criptografia evita vazamento de dados.
- Conformidade evita penalidades e aumenta a confiança.

Instrução:

- Pesquisar uma falha de segurança pública em nuvem.
- Identificar o erro e como poderia ter sido evitado.
- Postar no GitHub um resumo com link da notícia.

Resumo da aula:

- Segurança na nuvem começa com boas práticas de IAM.
- Criptografia deve ser sempre ativada.
- Conformidade protege usuários e empresas.

Próxima aula: Desenvolvimento de uma aplicação baseada em nuvem.

Objetivos da Aula:

- Compreender os fundamentos de segurança na nuvem usando Azure.
- Trabalhar com controle de acesso, criptografia e conformidade.
- Aplicar boas práticas com ferramentas nativas da Microsoft Azure.

Problemas recorrentes:

- Chaves de acesso expostas em código-fonte.
- Armazenamento com permissões públicas.
- Acesso excessivo a recursos críticos.
- Falta de criptografia de dados sensíveis.

RBAC - Role Based Access Control:

- Controle baseado em funções (Reader, Contributor, Owner).
- Aplicado a usuários, grupos, ou identidades gerenciadas.

Boas práticas:

- Atribuir a menor permissão necessária.
- Evitar uso do proprietário global para tarefas do dia a dia.
- Auditar periodicamente os acessos.

Demonstração: Criando um Acesso com RBAC

Passos:

- 1 Acessar Azure Portal.
- 2 Escolher um recurso (ex: Storage Account).
- 3 Ir em "Controle de Acesso (IAM)" > "Adicionar função".
- 4 Escolher função: Storage Blob Data Reader.
- 5 Atribuir a um usuário específico.

Criptografia em repouso:

- Ativada por padrão com chaves gerenciadas pela Microsoft.
- Pode ser configurada para usar Azure Key Vault.

Criptografia em trânsito:

- TLS habilitado por padrão nos serviços.
- Deve ser reforçada com uso de certificados próprios e HTTPS.

Key Vault:

- Gerencia chaves, segredos e certificados com segurança.

Demonstração: Criptografia com Azure Key Vault

Passos:

- 1 Criar um Azure Key Vault.
- 2 Adicionar uma nova chave simétrica.
- 3 Configurar um serviço (Storage ou SQL Database) para usar a chave.
- 4 Monitorar o uso da chave com logs.

Ferramentas da Azure:

- **Microsoft Defender for Cloud:** Avalia e protege recursos.
- **Azure Policy:** Garante conformidade com regras de segurança.
- **Compliance Manager:** Avaliação contínua da conformidade (ISO, LGPD, GDPR).

Exemplos de padrões:

- ISO 27001, LGPD, GDPR, NIST, CIS Benchmarks.

Tarefa:

- Criar uma Storage Account.
- Restringir acesso com RBAC.
- Ativar criptografia com chave personalizada do Key Vault.
- Documentar as ações e justificar escolhas.

Pergunta:

- O que você faria diferente em relação à segurança do seu ambiente de nuvem hoje?

Pontos de discussão:

- Evitar permissões amplas.
- Usar criptografia gerenciada.
- Monitorar atividades com Defender e Logs.

Instrução:

- Escolher um caso real de vazamento de dados na nuvem (de preferência em Azure).
- Explicar o que causou o problema e como poderia ter sido evitado.
- Subir um documento no GitHub pessoal com link da notícia.

Resumo da aula:

- Segurança na Azure envolve RBAC, Key Vault e políticas de conformidade.
- A criptografia deve ser tratada como padrão, não como opção.
- Segurança deve ser pensada desde o início do projeto.

Próxima aula: Desenvolvimento de uma aplicação baseada em nuvem.

Objetivos da Aula:

- Compreender os principais elementos de uma aplicação em nuvem.
- Projetar e implementar uma aplicação simples utilizando serviços em nuvem.
- Integrar recursos como computação, banco de dados e armazenamento.

Componentes comuns:

- Backend: lógica da aplicação (APIs, processamento).
- Frontend: interface com o usuário.
- Banco de dados: relacional ou NoSQL.
- Armazenamento de arquivos: buckets (Blob/S3).
- Autenticação: identidade e controle de acesso.

Para esta aula usaremos:

- **FastAPI (Python)** para a API REST.
- **SQLite** ou **PostgreSQL** como banco de dados.
- **Azure App Service** ou **AWS Elastic Beanstalk** para deploy.
- **Storage:** Azure Blob Storage ou AWS S3 para arquivos.

Passo a Passo do Projeto

1. Criar API básica com FastAPI

- Endpoints de cadastro e listagem.

2. Conectar com banco de dados

- SQLite local ou RDS/PostgreSQL gerenciado.

3. Upload de arquivos

- Salvar arquivos no Storage (Blob/S3).

4. Deploy

- Subir aplicação para nuvem (App Service ou EBS).

Demonstração: FastAPI Simples

Exemplo básico:

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/")
6 def home():
7     return {"msg": "API na nuvem"}
```

Exemplo com SQLite:

```
1 import sqlite3
2
3 conn = sqlite3.connect("dados.db")
4 cursor = conn.cursor()
5 cursor.execute("CREATE TABLE IF NOT EXISTS usuarios (id
6                 INTEGER, nome TEXT)")
7 conn.commit()
```

Caminhos possíveis:

- Azure: Blob Storage com chave de acesso.
- AWS: S3 com política pública ou assinada.

Prática:

- Criar bucket.
- Subir arquivo da API.
- Retornar URL acessível.

Azure App Service:

- Subir código via GitHub ou FTP.
- Configurar runtime (Python 3.x).

AWS Elastic Beanstalk:

- Criar ambiente com Docker ou nativo.
- Subir com CLI ou zip do projeto.

Tarefa:

- Criar um projeto FastAPI com endpoint de cadastro.
- Salvar dados em banco (local ou gerenciado).
- Fazer upload de imagem e retornar link público.
- Fazer o deploy da aplicação na nuvem.

Pergunta:

- Quais vantagens você percebe em desenvolver diretamente na nuvem?

Discussão:

- Agilidade no deploy e atualização.
- Integração nativa com banco, storage e logs.
- Escalabilidade e segurança embutidas.

Instrução:

- Finalizar o projeto local.
- Documentar a aplicação (README).
- Subir o projeto completo no GitHub com link de acesso.

Resumo da aula:

- Desenvolvemos uma aplicação completa na nuvem.
- Utilizamos FastAPI, banco de dados e storage.
- Publicamos a aplicação para acesso externo.

Próxima aula: Integração de serviços na nuvem para otimização de aplicações.

Objetivos da Aula:

- Construir uma aplicação web simples e hospedá-la na Azure.
- Utilizar App Service, Azure Storage e Azure Database.
- Entender a arquitetura de aplicações cloud-native.

Componentes:

- Azure App Service (backend FastAPI).
- Azure SQL Database (armazenamento relacional).
- Azure Blob Storage (arquivos estáticos).
- Azure Monitor (logs e métricas).

- **FastAPI (Python)** - API web leve e eficiente.
- **Azure App Service** - Hospedagem gerenciada.
- **Azure SQL** - Banco de dados relacional na nuvem.
- **Blob Storage** - Upload e leitura de arquivos.

1. Criar API com FastAPI

- Endpoints para cadastro de dados e upload.

2. Conectar com Azure SQL Database

- Usar biblioteca pyodbc ou sqlalchemy.

3. Integrar com Azure Blob Storage

- Upload de arquivos com autenticação via chave de acesso.

4. Deploy via App Service

- Usar o Azure CLI ou GitHub Actions para publicação.

Exemplo de API com FastAPI

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/")
6 def home():
7     return {"mensagem": "API executando na Azure"}
```

Exemplo com pyodbc:

```
1 import pyodbc
2
3 conn = pyodbc.connect(
4     'DRIVER={ODBC Driver 17 for SQL Server};'
5     'SERVER=nome.database.windows.net;'
6     'DATABASE=meubanco;UID=usuario;PWD=senha'
7 )
```

Upload com Azure Blob Storage

Exemplo usando SDK Python:

```
1 from azure.storage.blob import BlobServiceClient
2
3 blob_service = BlobServiceClient.from_connection_string(
4     conn_str)
5
6 container_client = blob_service.get_container_client("
7     imagens")
8
9 with open("foto.png", "rb") as data:
10     container_client.upload_blob(name="foto.png", data=data)
```

Deploy com Azure App Service

Etapas:

- 1 Criar recurso “App Service” no portal Azure.
- 2 Selecionar runtime Python 3.11.
- 3 Fazer push via GitHub ou Azure CLI:
- 4 `az webapp up -name nome-do-app -runtime "PYTHON|3.11"`

Tarefa:

- Criar um pequeno sistema com FastAPI.
- Realizar persistência em Azure SQL.
- Upload de arquivos para Blob Storage.
- Publicar a aplicação usando App Service.

Práticas recomendadas:

- Usar Azure Key Vault para segredos (strings de conexão).
- Configurar CORS e HTTPS obrigatório.
- Monitorar com Azure Application Insights.

Pergunta:

- O que você aprendeu ao montar uma aplicação completa na Azure?

Discussão:

- Redução de complexidade com serviços gerenciados.
- Facilidade de integração e escalabilidade.
- Visão prática de arquitetura cloud-native.

Instrução:

- Finalizar o projeto e fazer deploy funcional na Azure.
- Subir código no GitHub com instruções no README.md.
- Compartilhar link do App Service funcionando.

Resumo da aula:

- Desenvolvemos uma aplicação integrada à Azure.
- Utilizamos App Service, SQL Database e Blob Storage.
- Aplicamos boas práticas de deploy e segurança.

Próxima aula: Integração de serviços na nuvem para otimização de aplicações.

Objetivos da Aula:

- Entender como integrar serviços da Azure em aplicações.
- Otimizar desempenho, escalabilidade e gerenciamento.
- Aplicar práticas modernas de arquitetura distribuída.

Por que Integrar Serviços?

Motivações:

- Reduzir carga no backend.
- Aumentar resiliência e disponibilidade.
- Automatizar processos com eficiência.

Exemplo prático:

- API → Azure SQL → Blob Storage → Notification → Monitoramento

- **Azure Functions** - processamento serverless.
- **Azure Service Bus** - fila para comunicação assíncrona.
- **Azure Logic Apps** - automação de fluxos com gatilhos.
- **Azure Monitor / Application Insights** - rastreamento e logs.
- **Azure API Management** - gateway para organizar APIs.

Exemplo: Aplicação FastAPI

- API recebe dados de usuário.
- Armazena em Azure SQL.
- Dispara Azure Function para processamento.
- Função salva log no Blob e envia mensagem para Service Bus.
- Logic App escuta o Service Bus e envia e-mail com status.

Uso típico:

- Responder a eventos (upload, novos registros).
- Automatizar tarefas pontuais.

Exemplo:

```
1 def main(req: func.HttpRequest) -> func.HttpResponse:  
2     name = req.params.get('name')  
3     return func.HttpResponse(f"Olá {name}")
```

Casos de uso:

- Notificações por e-mail/SMS.
- Integração com GitHub, Outlook, SharePoint.
- Processos automáticos entre sistemas.

Gatilhos comuns:

- Novo item no Service Bus.
- Upload em Blob Storage.

Finalidade:

- Acompanhar performance da aplicação.
- Diagnosticar erros em tempo real.
- Visualizar telemetria e uso de recursos.

Prática:

- Ativar Application Insights no App Service.
- Visualizar logs no portal e configurar alertas.

Tarefa em duplas:

- Criar Azure Function que recebe dados de API.
- Gravar log da função em Blob.
- Publicar status da função em Service Bus.
- Criar Logic App para escutar e enviar e-mail.

Pergunta:

- Qual a principal vantagem de integrar serviços ao invés de codificar tudo?

Discussão:

- Economia de tempo e recursos.
- Alta disponibilidade e escalabilidade embutidas.
- Menor complexidade no backend da aplicação.

Instrução:

- Escolher dois serviços Azure e documentar uma possível integração.
- Descrever o fluxo de dados entre eles.
- Subir a documentação com esquema no GitHub.

Resumo da aula:

- Integração entre serviços facilita escalabilidade e automação.
- Azure Functions, Logic Apps e Service Bus são chaves.
- Otimização vem da composição de serviços, não do código isolado.

Próxima aula: Implementação de alta disponibilidade e escalabilidade.

Objetivos da Aula:

- Integrar serviços nativos da Azure em uma aplicação web.
- Melhorar desempenho, escalabilidade e automação.
- Criar soluções desacopladas e resilientes.

Por que Integrar Serviços na Azure?

Vantagens:

- Reduz complexidade no código.
- Automatiza tarefas e respostas a eventos.
- Escala horizontalmente com menos esforço.
- Garante maior confiabilidade da aplicação.

- **Azure Functions** – lógica sob demanda (serverless).
- **Azure Logic Apps** – automação de fluxos sem código.
- **Azure Service Bus** – mensagens assíncronas entre serviços.
- **Azure Event Grid** – eventos entre recursos e aplicações.
- **Azure Blob Storage** – armazenamento de arquivos.
- **Azure Application Insights** – telemetria e rastreamento.

Exemplo: Aplicação de Envio de Documentos

- Usuário envia formulário via API.
- Azure Function processa os dados e armazena no Blob.
- Azure Event Grid dispara evento para o Service Bus.
- Logic App escuta fila e envia e-mail de confirmação.
- Application Insights rastreia toda a operação.

Vantagens:

- Custo sob demanda.
- Escala automática.
- Integração com outros serviços via bindings.

Exemplo:

```
1 import logging
2 import azure.functions as func
3
4 def main(req: func.HttpRequest) -> func.HttpResponse:
5     nome = req.params.get('nome')
6     return func.HttpResponse(f"Olá, {nome}")
```

Para quê usar:

- Automatizar notificações e processos.
- Conectar APIs, SaaS, bancos de dados.

Gatilhos comuns:

- Nova mensagem no Service Bus.
- Novo arquivo no Blob Storage.

Utilização:

- Comunicação assíncrona e confiável entre microserviços.
- Filas desacoplam a API do processamento.

Cenário:

- Azure Function escreve na fila.
- Logic App consome e dispara automações.

Funções:

- Rastrear exceções e requisições HTTP.
- Ver tempo de resposta e gargalos.
- Consultas KQL no portal Azure.

Prática:

- Ativar Insights no App Service ou Function App.
- Visualizar métricas no painel do Azure Monitor.

Em duplas:

- Criar Function App que recebe dados de uma API.
- Salvar o dado em Blob Storage.
- Enviar uma mensagem para o Service Bus.
- Criar Logic App que envia e-mail com os dados recebidos.

Pergunta:

- Como a integração de serviços na Azure reduz complexidade?

Discussão:

- Menos código, mais conectividade.
- Redução de erros humanos por automação.
- Aumenta observabilidade com insights.

Instrução:

- Escolha 3 serviços da Azure.
- Monte um fluxo de integração e explique cada passo.
- Publique a ideia no GitHub com esquema visual (pode ser desenhado).

Resumo da aula:

- Integração reduz complexidade e melhora escalabilidade.
- Azure Functions + Logic Apps + Service Bus = combinação poderosa.
- Application Insights garante visibilidade e controle.

Próxima aula: Implementação de alta disponibilidade e escalabilidade.

Objetivos da Aula:

- Compreender os conceitos de alta disponibilidade e escalabilidade.
- Implementar recursos nativos da Azure que garantem esses aspectos.
- Aplicar práticas que garantem resiliência em aplicações na nuvem.

O que é Alta Disponibilidade (HA)?

Definição:

- Capacidade da aplicação se manter disponível mesmo diante de falhas.

Práticas comuns:

- Instâncias em múltiplas zonas ou regiões.
- Balanceamento de carga.
- Monitoramento e failover automático.

O que é Escalabilidade?

Definição:

- Capacidade de aumentar ou reduzir recursos conforme a demanda.

Tipos:

- **Vertical:** aumentar capacidade de uma instância.
- **Horizontal:** adicionar mais instâncias.
- **Automática (Auto Scale):** escalar baseado em métricas.

Recursos nativos:

- Azure Load Balancer – balanceamento de tráfego.
- Azure App Service Plan – escala automática integrada.
- Azure Virtual Machine Scale Set (VMSS).
- Azure Front Door – distribuição global com failover.
- Availability Zones e Availability Sets.

Configuração:

- Plano do tipo “Standard” ou superior.
- Métricas usadas: uso de CPU, memória ou requisições.
- Define número mínimo e máximo de instâncias.

Exemplo:

- Escalar horizontalmente de 1 até 5 instâncias com CPU acima de 70%.

Benefícios:

- Distribui tráfego entre múltiplas regiões.
- Detecta falhas e redireciona o tráfego.
- Inclui cache, WAF e regras de roteamento inteligente.

Cenário típico:

- Aplicação hospedada no Brasil e backup na Europa.
- Em caso de falha, tráfego é redirecionado automaticamente.

Diferenças:

- **Availability Zone:** data centers fisicamente separados.
- **Availability Set:** separação lógica de VMs no mesmo data center.

Uso:

- Melhor usado em máquinas virtuais e bancos gerenciados (como SQL).
- Garante que uma falha física não afete todas as instâncias.

Tarefa individual:

- Criar um App Service com escalabilidade automática.
- Configurar regra de autoescala por uso de CPU.
- Criar uma regra de alertas via Azure Monitor.
- Documentar no GitHub com prints e explicações.

Pergunta:

- O que pode impactar negativamente a disponibilidade de um sistema mesmo com nuvem?

Discussão:

- Falta de planejamento para falhas.
- Código mal escrito sem controle de erros.
- Recursos mal configurados (ex: sem zona de redundância).

Instrução:

- Estudar o comportamento de escalabilidade de um App Service criado hoje.
- Testar manualmente com locust ou ab.
- Anotar como a aplicação reagiu e subir relatório no GitHub.

Resumo da aula:

- Alta disponibilidade e escalabilidade são essenciais em produção.
- Azure fornece ferramentas nativas como App Service Plan, Load Balancer, Front Door e Scale Sets.
- Configurações adequadas garantem resiliência e performance.

Próxima aula: Automação e otimização de custos na nuvem.

Objetivos da Aula:

- Compreender estratégias para reduzir custos na nuvem.
- Automatizar tarefas repetitivas com ferramentas da Azure.
- Monitorar o uso e identificar desperdícios de recursos.

Por que Otimizar Custos?

Motivações principais:

- Evitar cobranças excessivas com recursos não utilizados.
- Tornar os projetos mais sustentáveis e eficientes.
- Liberar orçamento para áreas estratégicas.

Ferramentas da Azure para Otimização

- **Azure Cost Management + Billing** – análise e controle de gastos.
- **Azure Advisor** – recomendações automáticas de economia.
- **Azure Automation** – scripts e runbooks para desligar/startar recursos.
- **Azure Reservations** – desconto para uso prolongado de serviços.
- **Resource Tags** – organização e categorização por custo/projeto.

Principais recursos:

- Painel de custos por serviço, grupo ou tempo.
- Alertas de orçamento e uso.
- Previsão de gastos com base no consumo atual.

Prática recomendada:

- Criar alertas para evitar estouro do orçamento mensal.

Categorias de recomendações:

- Custo – desligar VMs inativas, reduzir planos não usados.
- Segurança – atualizar certificados e regras de firewall.
- Performance – balancear carga e escalar corretamente.

Como usar:

- Acessar portal Azure > Advisor.
- Aplicar as recomendações sugeridas.

Recursos:

- Criar **runbooks** com scripts PowerShell ou Python.
- Agendar tarefas como desligar VMs fora do expediente.

Exemplo prático:

- Script que desliga instâncias toda sexta às 20h.
- Script que inicia novamente segunda às 8h.

Azure Reservations:

- Desconto de até 72% por compromissos de 1 a 3 anos.
- Ideal para VMs, bancos e App Service em uso constante.

Spot Instances:

- Recursos com custo muito baixo.
- Boa opção para cargas não críticas ou temporárias.

Uso de tags:

- Permite agrupar e classificar por projeto, dono, ambiente.
- Facilita análises de custo por equipe ou aplicação.

Exemplo de tags:

- `project: sistema-contabil, env: production, owner: roni`

Tarefa em grupo:

- Acessar Azure Advisor e listar 3 recomendações de economia.
- Criar um runbook para desligar um recurso automaticamente.
- Criar um alerta de custo mensal com limite de R\$ 100.
- Documentar no GitHub com prints e explicações.

Pergunta:

- Sua aplicação atual consome mais recursos do que precisa?

Discussão:

- Qualquer recurso ativo consome orçamento.
- Automatizar e monitorar são chaves para economizar.

Instrução:

- Estudar e aplicar um runbook no Azure Automation.
- Criar um recurso qualquer, programar desligamento automático.
- Subir a documentação no GitHub com comandos utilizados.

Resumo da aula:

- Azure oferece ferramentas completas para controle de custos.
- Automatizar desligamentos e reservas reduz muito os gastos.
- Tagueamento e alertas ajudam na organização e prevenção.

Próxima aula: Estudo de caso: análise de arquiteturas Cloud Computing reais.

Objetivos da Aula:

- Analisar arquiteturas reais implementadas em Azure.
- Entender decisões de design para alta performance e segurança.
- Avaliar componentes usados e alternativas possíveis.

O Que Observar em uma Arquitetura

Critérios principais:

- Alta disponibilidade e escalabilidade.
- Divisão por camadas (frontend, backend, dados).
- Segurança e controle de acesso.
- Monitoramento, logging e recuperação.
- Custo e simplicidade de manutenção.

Estudo de Caso 1: Sistema de E-commerce

Descrição:

- Plataforma de vendas com pagamento e estoque.

Componentes usados:

- Azure App Service (frontend/backend)
- Azure SQL Database
- Azure Blob Storage (imagens de produtos)
- Azure CDN + Front Door
- Azure Monitor + Application Insights

Estudo de Caso 1: Pontos de Destaque

Boas práticas:

- Front Door com failover entre regiões.
- Escalabilidade automática do App Service.
- Chaves armazenadas no Azure Key Vault.

Oportunidades de melhoria:

- Implementar fila com Azure Service Bus.
- Automatizar tarefas com Azure Functions.

Estudo de Caso 2: Aplicação de Telemetria IoT

Descrição:

- Dispositivos enviam dados em tempo real.

Componentes usados:

- Azure IoT Hub
- Azure Stream Analytics
- Azure Data Lake + Synapse Analytics
- Power BI para dashboards
- Azure Functions para ações automatizadas

Boas práticas:

- Processamento em tempo real com Stream Analytics.
- Armazenamento otimizado em Data Lake.
- Visualização com Power BI em tempo quase real.

Oportunidades de melhoria:

- Criar alertas com Azure Monitor.
- Usar Azure Logic Apps para integrações externas.

Tarefa:

- Escolher um sistema real ou fictício.
- Desenhar uma arquitetura cloud-native na Azure.
- Justificar escolha de cada serviço.
- Apresentar o desenho para a turma.

Sugestões:

- Microsoft Azure Architecture Center
- Azure Diagram Tool (diagrams.net com ícones Azure)
- Ferramentas alternativas: Lucidchart, Excalidraw, Whimsical

Pergunta:

- O que mais impacta a decisão entre um serviço e outro na Azure?

Discussão:

- Custo vs. desempenho.
- Complexidade vs. escalabilidade.
- Segurança vs. facilidade de operação.

Instrução:

- Criar e documentar no GitHub uma arquitetura Azure para:
 - Um sistema de controle de agendamentos.
- Incluir desenho, lista de serviços e justificativas.

Resumo da aula:

- Estudamos arquiteturas reais com foco em Azure.
- Avaliamos boas práticas e pontos de melhoria.
- Criamos esboços de soluções com propósito real.

Próxima aula: Desenvolvimento do projeto final.

Objetivos da Aula:

- Iniciar a construção prática do projeto final.
- Aplicar os conceitos vistos durante o semestre.
- Utilizar recursos da nuvem Azure em uma solução real.

Proposta do Projeto Final

Tema:

- Criar uma aplicação funcional baseada em nuvem com pelo menos:
 - Backend (API ou função).
 - Banco de dados (relacional ou NoSQL).
 - Armazenamento de arquivos.
 - Deploy na Azure.

Exemplos:

- Sistema de agendamento online.
- Catálogo de produtos com upload de imagens.
- Aplicação de análise de dados.

O que será observado:

- Uso correto dos serviços Azure.
- Estrutura e organização do projeto.
- Boas práticas de segurança e escalabilidade.
- Documentação clara no GitHub.
- Apresentação objetiva e funcional.

Cada projeto deve conter:

- Azure App Service ou Azure Functions.
- Azure SQL Database ou Cosmos DB.
- Azure Blob Storage.
- Application Insights habilitado.
- Controle de acesso e autenticação mínima (ex: API key).

- Código versionado no GitHub.
- README com instruções de execução.
- Link da aplicação funcionando na Azure.
- Prints das telas e arquitetura utilizada.
- Organização em módulos e pastas.

Planejamento das próximas aulas:

- Aula 1: Definir tema e montar arquitetura.
- Aula 2: Criar repositório, iniciar o backend.
- Aula 3: Banco de dados e storage.
- Aula 4: Deploy e ajustes finais.
- Aula 5: Apresentação e feedback.

Meta da aula:

- Definir o tema do projeto.
- Criar o repositório no GitHub.
- Criar o recurso principal na Azure (App Service ou Function App).
- Esboçar o diagrama da arquitetura.

Pergunta:

- Qual recurso da Azure você pretende explorar mais no projeto?

Discussão:

- Cada grupo pode focar em uma área (monitoramento, automação, integração).
- A ideia é aplicar na prática e documentar a experiência.

Instrução:

- Finalizar o setup do repositório.
- Criar a estrutura mínima do projeto.
- Subir um README com: título, objetivo, tecnologias e cronograma do grupo.

Resumo da aula:

- Iniciamos oficialmente o projeto final.
- Definimos estrutura mínima e tecnologias obrigatórias.
- Próximas aulas serão práticas com foco total no desenvolvimento.

Próxima aula: Continuação do desenvolvimento com foco no backend e banco de dados.